

Quantum Classifiers Against Classical Classifiers in Machine Learning and Deep Learning

Ryan Li
COMP 360 Final Report
Spring 2026

May 12, 2026

1 Problem Statement

The central problem this project addresses is empirical: under what conditions, if any, does a small parameterised quantum circuit trained as a supervised classifier compete with standard classical classifiers? A variational quantum classifier (VQC) is a parameterised quantum circuit $U(\boldsymbol{\theta}, \mathbf{x})$ acting on n qubits, applied to an initial state $|0\rangle^{\otimes n}$, followed by a measurement whose statistics are interpreted as a class probability distribution; the parameters $\boldsymbol{\theta}$ are trained by a classical optimiser on a labelled dataset [1, 2, 3]. A natural alternative is the quantum-kernel support-vector machine (QSVC), which uses the inner products $\langle \phi(\mathbf{x}_i) | \phi(\mathbf{x}_j) \rangle$ between data states $|\phi(\mathbf{x})\rangle = U_\phi(\mathbf{x})|0\rangle^{\otimes n}$ as a kernel for a classical SVM [1, 4].

We compare these two quantum approaches against three classical baselines (logistic regression, RBF-SVM, and a multilayer perceptron whose parameter count is matched to the VQC’s) along four axes drawn directly from the proposal: **accuracy** (Research Question 1), **architecture choice** (RQ2), **trainability** via the barren-plateau phenomenon (RQ3), and **noise robustness** (RQ4).

2 Why the Problem Matters

Quantum machine learning (QML) sits at the intersection of two of the most active areas in computing. On one side, near-term quantum hardware operates in what Preskill termed the noisy intermediate-scale quantum (NISQ) regime [5]: 50–1000 qubits, no error correction, gate fidelities around 99–99.9%, and short coherence times. Algorithms that can be useful on NISQ devices must therefore be hybrid quantum-classical: the quantum subroutine evaluates an expectation or a kernel, and a classical optimiser drives the parameters.

On the other side, modern machine learning is dominated by high-dimensional parametric models trained by gradient descent. A parameterised quantum circuit is in some respects a natural quantum analogue: it is a unitary map $U(\boldsymbol{\theta})$ that defines a function class, the function class can be searched by gradient (via the parameter-shift rule [3]) or gradient-free methods, and the state space has 2^n amplitudes for n qubits, suggesting an exponentially large representational capacity. Whether this exponential capacity translates into a real machine-learning advantage is the central open question of the field.

The question matters for three reasons. First, it is one of the few near-term quantum problems that can be experimentally addressed without fault tolerance — VQC circuits are by construction shallow. Second, the answer informs whether continued investment in QML libraries (Qiskit

Machine Learning, PennyLane, TensorFlow Quantum) is justified. Third, it speaks to a deeper theoretical issue: classical machine learning succeeded in part because of the surprising *trainability* of overparameterised neural networks; the analogous quantum result is severely complicated by the barren-plateau phenomenon [7, 8, 9], which has no classical analogue and which limits the depth of any practically trainable quantum model.

3 State of the Art

Havlíček et al. (2019) [1] introduced the QSVC framework on IBM Q hardware and demonstrated 100% test accuracy on a synthetic dataset constructed to be classically hard via the ZZFeatureMap. This result is the most-cited piece of empirical evidence for a quantum advantage in supervised learning, but its scope is narrow: the dataset was constructed to make the quantum kernel optimal. Schuld et al. (2020) [2] formalised the VQC as a “circuit-centric” classifier with theoretical capacity arguments; Mitarai et al. (2018) [3] introduced the parameter-shift rule that makes gradient-based training of VQCs feasible.

Subsequent work has been more cautious. McClean et al. (2018) [7] showed that random parameterised circuits exhibit *barren plateaus*: the variance of the loss gradient decays exponentially with the number of qubits, so that after random initialisation a circuit of moderate depth cannot be trained by any gradient method. Cerezo et al. (2021) [8] refined this: plateaus depend on both circuit expressivity and the locality of the cost function, and shallow circuits with local cost functions can escape them. Larocca et al. (2025) [9] surveyed the current understanding: plateaus are now a unifying lens through which much of QML must be evaluated. Grant et al. (2019) [10] proposed an identity-block initialisation that places the circuit near the identity at initialisation, mitigating the plateau in practice.

Most recently, several large-scale empirical benchmarks have found that VQCs rarely beat classical models of comparable parameter count on generic tasks, while quantum-kernel methods occasionally match classical kernels. Cerezo et al. (2021) [6] reviewed variational quantum algorithms more broadly; the picture is one of cautious optimism for problem-specific quantum kernels and pessimism for generic VQCs without inductive bias.

Our experiments aim to characterise these trade-offs with experiments and comparison between training results.

4 Core Algorithmic Ideas

4.1 The variational quantum classifier

A VQC is built from three pieces:

1. A **feature map** $U_\phi(\mathbf{x})$ that encodes a classical input $\mathbf{x} \in \mathbb{R}^d$ into a quantum state $|\phi(\mathbf{x})\rangle = U_\phi(\mathbf{x})|0\rangle^{\otimes n}$.
2. A **trainable ansatz** $U(\boldsymbol{\theta})$ with classical parameters $\boldsymbol{\theta} \in \mathbb{R}^p$.
3. A **measurement** of a Hermitian observable, typically a single-qubit Pauli-Z. The measurement statistics are mapped to a predicted class label.

For an input $\mathbf{x}$ and parameters $\boldsymbol{\theta}$ the full state is

$$|\psi(\mathbf{x}, \boldsymbol{\theta})\rangle = U(\boldsymbol{\theta}) U_\phi(\mathbf{x}) |0\rangle^{\otimes n}.$$

A loss function $L(\boldsymbol{\theta}) = \frac{1}{N} \sum_i \ell(y_i, \hat{y}(\mathbf{x}_i; \boldsymbol{\theta}))$ is then minimised by a classical optimiser. The two we tested, COBYLA and SPSA, are both gradient-free: COBYLA builds a linear approximation of L in a neighbourhood and uses linear-programming steps, while SPSA estimates gradients by random simultaneous perturbations, making it more robust to stochastic noise from shot-based simulation.

4.2 Feature maps

We implemented three feature maps:

Angle encoding: $U_\phi(\mathbf{x}) = \bigotimes_i R_y(x_i)$ — a single R_y rotation per qubit, the simplest map and classically simulable in closed form. Its expressivity is bounded by sin and cos on each axis.

ZZFeatureMap [1]: alternating Hadamards, single-qubit Z rotations $R_z(x_i)$, and two-qubit ZZ rotations $\exp(-ix_i x_j Z_i Z_j / 2)$. This map is conjectured to be hard to simulate classically, and is the canonical “quantum-enhanced” encoder in the literature.

Data re-uploading: the input is interleaved with trainable gates and re-encoded multiple times [11]; this increases the effective expressivity without adding qubits.

4.3 Ansätze

We tested two hardware-efficient ansätze from the Qiskit library:

RealAmplitudes: alternating R_y rotations on every qubit followed by a layer of CNOT entanglers; produces real-valued amplitudes only.

EfficientSU2: alternating R_y, R_z rotations followed by configurable CNOT entanglers (linear, circular, or full topology); spans the full single-qubit $SU(2)$ at each layer.

4.4 Quantum kernel

For QSVC, the kernel matrix entries are

$$K_{ij} = |\langle \phi(\mathbf{x}_i) | \phi(\mathbf{x}_j) \rangle|^2 = |\langle 0|^{\otimes n} U_\phi^\dagger(\mathbf{x}_i) U_\phi(\mathbf{x}_j) |0\rangle^{\otimes n}|^2.$$

This can be estimated on a quantum computer by preparing $U_\phi^\dagger(\mathbf{x}_i) U_\phi(\mathbf{x}_j) |0\rangle$ and counting the all-zero outcome. The resulting kernel is fed into a standard classical SVM. This decouples “quantum feature space” from “quantum trainable circuit”, which is a distinction often blurred in the literature.

4.5 The parameter-shift rule and gradient variance

For a parameter θ_k appearing as $\exp(-i\theta_k G/2)$ with $G^2 = I$ (e.g. a Pauli), the exact gradient of the expectation of any observable O is

$$\frac{\partial \langle O \rangle}{\partial \theta_k} = \frac{1}{2} [\langle O \rangle_{\theta_k + \pi/2} - \langle O \rangle_{\theta_k - \pi/2}].$$

Two circuit evaluations give the exact gradient with respect to one parameter. We used this rule in our barren-plateau probe (Section 7.3) to compute $\partial L / \partial \theta_0$ at random parameter points.

4.6 Multi-class measurement interpretation

For binary classification the predicted probability is $p(1|\mathbf{x}) = \frac{1 + \langle Z_0 \rangle}{2}$. For C classes, the standard recipe (used in Qiskit’s **SamplerQNN**) interprets each measurement outcome $k \in \{0, \dots, 2^n - 1\}$ as a class via a mapping $f : \{0, \dots, 2^n - 1\} \rightarrow \{0, \dots, C - 1\}$, then aggregates probabilities. The default

parity map $f(k) = k \bmod C$ creates collisions when 2^n is not a multiple of C : with 2 qubits and 3 classes, outcomes 00 and 11 both map to class 0, capping accuracy regardless of training. Our implementation uses a direct mapping ($f(k) = k$ for $k < C$) when $n \geq \lceil \log_2 C \rceil$, which fixes this issue on Iris.

5 Ideas from Class

This project drew on most of the foundational material from class, applied at a higher level of abstraction:

Qubits and quantum states. The state space of n qubits is $(\mathbb{C}^2)^{\otimes n}$, with computational-basis kets $|x\rangle$ for $x \in \{0, 1\}^n$. Every VQC state is a unit vector in this space; for 6 qubits (used in our barren-plateau experiments) the state space has $2^6 = 64$ complex amplitudes. The dimension grows exponentially in n .

Quantum gates. Every operation in our circuits is built from the universal gate set we studied. The feature maps use Hadamards (H), $R_y(\theta)$ and $R_z(\theta)$ single-qubit rotations, and $\exp(-i\theta Z \otimes Z/2)$ two-qubit rotations. The ansätze use R_y, R_z and CNOT (CX) gates; in particular, RealAmplitudes is a literal chain of R_y rotations alternated with CNOTs.

Quantum circuits and measurement. The VQC is a sequential quantum circuit terminating in projective measurement in the computational basis. Following the partial-measurement formalism, when we measure a single-qubit observable $\langle Z_0 \rangle$ on an n -qubit state, we are effectively tracing out qubits 1 through $n - 1$ and computing $\langle Z \rangle$ on the resulting one-qubit reduced state. The probability of outcome 0 on qubit 0 equals $\sum_{x_1, \dots, x_{n-1}} |\langle 0 x_1 \dots x_{n-1} | \psi \rangle|^2$, giving the binary class probability used as the loss target.

Composite states and entanglement. The CNOT layers in the ansatz produce genuinely entangled multipartite states. In our 6-qubit runs, RealAmplitudes at depth 8 generates states with non-trivial entanglement across all bipartitions; this is the mechanism by which the circuit explores a high-dimensional Hilbert space, and also one cause of the barren-plateau phenomenon.

The fidelity test in quantum kernels. The QSVC kernel $K_{ij} = |\langle \phi(\mathbf{x}_i) | \phi(\mathbf{x}_j) \rangle|^2$ is estimated using the same overlap test we saw for distinguishing quantum states: prepare $U_\phi^\dagger(\mathbf{x}_i) U_\phi(\mathbf{x}_j) |0\rangle^{\otimes n}$ and measure the all-zero outcome probability.

6 Implementation and Methodology

6.1 Implementation stack

All experiments were implemented in Python 3.12 using Qiskit 2.4.1 with `qiskit-machine-learning` 0.9.0, `qiskit-algorithms` 0.4.0, `qiskit-aer` 0.17.2, and `qiskit-ibm-runtime` 0.46.1 for FakeBackend snapshots. Classical baselines used scikit-learn 1.8.0. All experiments are reproducible: every figure and number in this report is generated by the two CLI-driven scripts `quantum_classifiers.py`

and `classical_classifiers.py` included in the `final/` directory of our project submission, with seeds fixed and complete hyperparameters recorded in CSV.

6.2 Datasets and protocol

We used three small classification benchmarks of increasing difficulty:

Two-moons: 300 samples generated by `sklearn.datasets.make_moons` with noise level $\sigma = 0.25$; binary classification of two interleaving half-circles.

Iris: 150 samples (full dataset), reduced to 3 features by PCA; three-class classification. PCA to 3 components was required (rather than the proposal’s 2) to give the VQC 3 qubits and so avoid a parity-collision ceiling on multi-class outputs.

MNIST 0-vs-1: 300 samples subsampled from the digits “0” and “1” in MNIST 784, reduced to 4 features by PCA; binary classification.

All datasets were split 70/15/15 (train/validation/test) with three fixed seeds 42, 123, 7. Every accuracy in this report is reported as mean $\pm$ sample standard deviation across these three seeds. For quantum classifiers the data was MinMax-scaled into $[0, \pi]$ before feeding to the feature map.

6.3 Like-for-like parameter matching

A central design principle from the proposal was to compare the VQC and the MLP on a like-for-like trainable-parameter basis. For each dataset and VQC configuration we recorded the ansatz’s parameter count p_q and set the MLP’s hidden width so that its total trainable parameter count p_c matched p_q as closely as possible. The matches achieved were: two-moons $p_q = 6$ versus $p_c = 7$; Iris $p_q = 9$ versus $p_c = 9$; MNIST $p_q = 24$ versus $p_c = 30$. Logistic regression and RBF-SVM serve as strong non-matched baselines.

6.4 The full experimental matrix

A total of 190 training runs across 70 distinct configurations were executed, requiring approximately 2.5 hours of CPU time on a laptop. The breakdown is: 45 runs for RQ1 (5 methods $\times$ 3 datasets $\times$ 3 seeds, plus QSVC variants on two-moons); 45 runs for RQ2 (36 single-seed sweep + 9 multi-seed best-of-each); 40 evaluations for RQ3 (2 ansätze $\times$ 5 qubit counts $\times$ 4 depths, with 300 random parameter draws per cell); and 54 runs for RQ4 (3 depths $\times$ 5 noise levels $\times$ 3 seeds, plus FakeBackend and SPSA comparisons). All runs use the same data preprocessing, splitting, and quantum sampler infrastructure.

7 Results

7.1 RQ1: Like-for-like accuracy

Table 1 summarises the headline accuracy comparison across all three datasets, with mean $\pm$ standard deviation across three seeds.

Table 1: RQ1 test accuracy (mean $\pm$ std, $n = 3$ seeds). VQC parameter counts are 6 (two-moons), 9 (Iris), 24 (MNIST). MLP target was matched to within $\pm 25\%$.

Dataset	LogReg	SVM-RBF	MLP	VQC	QSVC (ZZ)
two-moons	0.837 ± 0.068	0.882 ± 0.064	0.733 ± 0.204	0.756 ± 0.118	0.748 ± 0.126
Iris (PCA-3)	0.942 ± 0.050	0.942 ± 0.066	0.841 ± 0.109	0.638 ± 0.025	0.667 ± 0.091
MNIST 0/1	0.993 ± 0.013	0.985 ± 0.026	0.978 ± 0.022	0.741 ± 0.013	0.933 ± 0.022

Three observations.

First, the VQC never wins. Its largest deficit is on MNIST (0.74 versus 0.99 for the best classical baseline) and the smallest on two-moons (0.76 versus 0.88). This is consistent with the broader literature finding [6, 9] that small trainable quantum models struggle to match classical models of equivalent parameter count.

Second, the QSVC is competitive on MNIST (0.933 versus 0.985 for SVM-RBF) but underperforms on Iris (0.667) and two-moons (0.748). The MNIST result is the most positive quantum finding in the project. To isolate the role of the feature map within the QSVC framework, we extended the proposal by running QSVC on two-moons with each of the three feature maps; the results are in Table 2.

Table 2: QSVC test accuracy on two-moons with different feature maps (mean $\pm$ std, $n = 3$ seeds). Angle and reupload encoding outperform the canonical ZZ feature map on this geometric task.

Feature map	Test accuracy
ZZFeatureMap (canonical)	0.748 ± 0.126
Angle encoding	0.844 ± 0.044
Data re-uploading (reps=2)	0.867 ± 0.080

Third, training accuracy on the VQC is also low (typically 0.64–0.78), ruling out overfitting as the explanation: the VQC is failing to fit even the training data. This points to the optimisation landscape — either limited expressivity from a too-shallow ansatz, or a hard-to-optimise loss surface. The barren-plateau analysis below addresses the second possibility.

7.2 RQ2: Architecture sweep on two-moons

To isolate the effect of feature map, depth, and entanglement topology on VQC performance, we ran a full sweep of $3 \times 4 \times 3 = 36$ configurations on two-moons at one seed. Table 3 gives the mean test accuracy averaged over entanglement topologies; Table 4 gives the mean averaged over depth.

Table 3: RQ2: mean test accuracy by feature map and depth on two-moons. Averaged over the three entanglement topologies. Angle encoding dominates; ZZ degrades at large depth.

Feature map	$L = 1$	$L = 2$	$L = 3$	$L = 4$	Mean
Angle	0.881	0.696	0.881	0.815	0.818
Reupload	0.681	0.733	0.689	0.726	0.707
ZZ	0.719	0.681	0.674	0.689	0.691

Table 4: RQ2: mean test accuracy by feature map and entanglement topology. Topology has a moderate effect; full entanglement performs best on the angle feature map.

Feature map	Circular	Full	Linear
Angle	0.789	0.828	0.839
Reupload	0.744	0.700	0.678
ZZ	0.683	0.717	0.672

When we rerun the best-of-each configuration with three seeds, the ranking is preserved with credible error bars (Table 5). The simpler angle feature map gives a 16-point edge over ZZ.

Table 5: RQ2: best feature-map / depth / entanglement combination per feature map, run with 3 seeds.

Feature map	Best config (depth, ent.)	Test accuracy
Angle	(2, full)	0.770 ± 0.084
Reupload	(2, linear)	0.726 ± 0.046
ZZ	(2, linear)	0.615 ± 0.071

These results tell a clear story. The simple angle encoding outperforms the canonical ZZFeatureMap by 15–20 percentage points on a small geometric dataset. The likely reason: ZZ produces a much higher-frequency function of the inputs, which on a benchmark whose decision boundary is two smooth half-circles is more curse than blessing. The result echoes a recent line of work suggesting that quantum advantage in supervised learning requires inductive bias matched to the data [12].

Entanglement topology has only a marginal effect (± 0.02 absolute), much smaller than the feature-map effect (± 0.15). This is consistent with the proposal’s hypothesis that “the choice of feature map dominates the architecture.”

7.3 RQ3: Barren plateaus

For each combination of $n \in \{2, 3, 4, 5, 6\}$ qubits and $L \in \{1, 2, 4, 8\}$ ansatz layers, we drew 300 random parameter vectors $\theta \sim \text{Uniform}([-\pi, \pi]^p)$ and computed $g_s = \partial \langle Z_0 \rangle / \partial \theta_0$ at each via the parameter-shift rule, then estimated $\text{Var}[g]$.

Table 6: RQ3: variance of $\partial L / \partial \theta_0$ across 300 random parameter samples (RealAmplitudes ansatz, linear entanglement). The variance decays approximately exponentially with n .

n qubits	$L = 1$	$L = 2$	$L = 4$	$L = 8$
2	1.057×10^{-1}	1.509×10^{-1}	2.037×10^{-1}	2.370×10^{-1}
3	6.720×10^{-2}	1.124×10^{-1}	1.013×10^{-1}	1.204×10^{-1}
4	2.894×10^{-2}	5.312×10^{-2}	7.201×10^{-2}	6.467×10^{-2}
5	1.142×10^{-2}	3.044×10^{-2}	3.270×10^{-2}	3.670×10^{-2}
6	6.685×10^{-3}	2.434×10^{-2}	2.022×10^{-2}	1.600×10^{-2}

Table 7: RQ3: log-linear fit of $\log \text{Var}[g]$ versus n , by ansatz and depth. A slope close to $-\ln 2 \approx -0.693$ means the variance halves every additional qubit.

Configuration	Slope	Halving period (qubits)	Predicts plateau?
RealAmplitudes, $L = 1$	-0.729	0.95	yes
RealAmplitudes, $L = 2$	-0.496	1.40	yes (weakly)
RealAmplitudes, $L = 4$	-0.575	1.21	yes
RealAmplitudes, $L = 8$	-0.658	1.05	yes
EfficientSU2, $L = 1$	-0.703	0.99	yes
EfficientSU2, $L = 2$	-0.637	1.09	yes
EfficientSU2, $L = 4$	-0.701	0.99	yes
EfficientSU2, $L = 8$	-0.707	0.98	yes

Every fit produces a negative slope of order -0.5 to -0.7 per qubit, the textbook signature of a barren plateau. The gradient variance halves roughly every additional qubit, exactly the regime McClean et al. [7] originally predicted. EfficientSU2’s plateau is slightly more pronounced (steeper slope across all depths), consistent with its larger expressivity — it covers more of $\text{SU}(2)$ per layer and so converges to a 2-design faster.

At fixed small n , the variance *increases* with depth before saturating (e.g. RealAmplitudes, $n = 2$: $0.106 \rightarrow 0.151 \rightarrow 0.204 \rightarrow 0.237$ as L goes $1 \rightarrow 8$). This is the depth-driven approach to a t -design described by Cerezo et al. [8]: shallow circuits do not yet fill Hilbert space evenly, so the gradient is biased and the variance is non-trivially structured. After depth saturates, the variance settles into the exponential decay in n .

The practical implication is concrete: at 6 qubits and $L = 4$, the expected gradient magnitude is already ~ 0.14 (compared with ~ 0.32 at 2 qubits). Extrapolating the fit suggests $n = 12$ would push the variance below 10^{-3} and any gradient-based optimiser would struggle. This explains why we keep our VQC experiments small.

7.4 RQ4: Noise robustness

We trained the angle-encoded, linear-entanglement VQC on two-moons under shot-based simulation (1024 shots) with three configurations of depolarising and readout noise, at three circuit depths $L \in \{2, 4, 8\}$. The 1-qubit depolarising rate p_1 was the swept variable; the 2-qubit rate was set to $10p_1$ and the symmetric readout-flip rate to $5p_1$, reflecting the typical ratio on IBM hardware. Test set size was 60 samples (1.7% granularity).

Table 8: RQ4: test accuracy by depth and noise level. Mean $\pm$ std over 3 seeds. Performance is essentially flat up to $p_1 = 0.005$, then degrades at $p_1 \geq 0.01$ across all depths.

L	Noiseless	$p_1=0.001$	$p_1=0.005$	$p_1=0.01$	$p_1=0.02$
2	0.806 ± 0.149	0.867 ± 0.029	0.872 ± 0.035	0.744 ± 0.127	0.800 ± 0.144
4	0.806 ± 0.149	0.800 ± 0.144	0.800 ± 0.159	0.806 ± 0.095	0.789 ± 0.136
8	0.872 ± 0.035	0.872 ± 0.035	0.861 ± 0.038	0.789 ± 0.084	0.761 ± 0.098

The picture is more nuanced than “noise makes everything worse.” At depth 2, light noise ($p_1 = 0.001$ – 0.005) actually *stabilises* training: the test accuracy mean rises from 0.806 (noiseless) to 0.867–0.872, and the variance shrinks by a factor of 4. This is a known regularisation effect of

shot noise on under-parameterised models. At $p_1 \geq 0.01$, the regularisation benefit is overwhelmed by the actual degradation of computed quantities, and accuracy drops by 8–12 percentage points.

At depth 8 the picture is cleaner: accuracy is stable through $p_1 = 0.005$ (0.87) and drops monotonically thereafter to 0.76 at $p_1 = 0.02$. The depth-2 anomaly is consistent with the literature finding [8] that shallow circuits in noisy settings sit near a phase boundary where small noise can help and large noise will eventually kill performance.

Table 9: RQ4 extensions: calibrated FakeBackend noise (IBM Brisbane snapshot) and SPSA versus COBYLA at depth 4, $p_1 = 0.005$. SPSA gives both higher mean accuracy and lower variance, consistent with its design for noisy gradient estimation.

Configuration	Test accuracy
Synthetic depolarising, $p_1=0.001$, $L=4$	0.800 ± 0.144
FakeBrisbane calibrated noise, $L=4$	0.800 ± 0.144
COBYLA at $p_1=0.005$, $L=4$	0.800 ± 0.159
SPSA at $p_1=0.005$, $L=4$	0.844 ± 0.025

The FakeBackend result is reassuring: the calibrated IBM Brisbane snapshot gives accuracy indistinguishable from our synthetic depolarising model at comparable error rates, suggesting that synthetic noise is a reasonable proxy for real device noise in this regime.

The SPSA-versus-COBYLA comparison is one of the project’s clearer findings: under shot noise, SPSA produces six-fold less variance in the final test accuracy (± 0.025 versus ± 0.159) at slightly higher mean. This is exactly the regime SPSA was designed for, and the result argues that in any real NISQ deployment, COBYLA should be replaced by SPSA.

8 Discussion

The four research questions collectively show a coherent picture. Classical methods win on like-for-like accuracy(RQ1), driven both by the VQC’s expressivity ceiling (low training accuracy) and by classical models’ decades of optimisation maturity. The VQC’s under-performance is mitigated but not erased when we switch to the quantum-kernel route: on MNIST the QSVC reaches 0.93, within reach of the classical 0.99. Architecture matters more than topology (RQ2), and on geometric data the simplest feature map dominates the canonical “quantum-enhanced” one. Barren plateaus are real and quantitative (RQ3): the gradient variance halves with each added qubit, and at our maximum depth the slope is essentially the $-\ln 2$ predicted by random-circuit arguments. Noise tolerance is better than naive intuition suggests (RQ4) up to about 1% two-qubit error, and SPSA outperforms COBYLA in the noisy regime.

The most striking practical lesson is that QML at the scales accessible to a course project is best thought of as a kernel method, not a neural-network analogue. The QSVC sidesteps the trainability problem entirely (no parameters to train on the quantum side), and the fidelity-test kernel is easy to evaluate on near-term hardware. The VQC, by contrast, has all the difficulties of training a small neural network plus a barren-plateau problem with no classical analogue.

9 What We Discovered

A short list, about what surprised us:

- **Single-seed numbers are nearly useless.** Our initial single-seed sweep gave VQC test accuracy of 0.73 on two-moons; the multi-seed redo gave 0.76 ± 0.12 . The first number was a fluke inside a wide distribution. Every accuracy in this report is multi-seed.
- **Implementation details have first-order effects.** The parity-mapping bug capped Iris accuracy at ~ 0.74 regardless of training quality; fixing it (using 3 qubits and a direct mapping) was necessary but not sufficient (Iris VQC still under-performs, but for substantive reasons now).
- **The barren-plateau prediction holds quantitatively.** A log-linear fit recovers slopes within 5% of the McClean prediction even on this small scale. This was the most satisfying experimental moment of the project.
- **The ZZFeatureMap is not always better.** The Havlíček paper [1] showed it could be perfect on a custom dataset; on generic geometric data we measure it to be the worst of three options. This is a tension worth thinking about: the very property that makes ZZ “quantum-enhanced” (high-frequency feature space) is what hurts it on a smooth decision boundary.
- **SPSA matters under noise.** A frequently overlooked detail.
- **Quantum simulation is slow.** The full RQ4 sweep took roughly 2 hours on a laptop CPU; a real hardware run with comparable shots would cost significant queue time on IBM Q. NISQ-era benchmarks will live or die by efficient simulation.

10 Future Work

Several directions follow naturally from this work.

Stronger data-encoding theory. Why does angle encoding beat ZZ on two-moons? The Kübler et al. result [12] suggests quantum kernels need inductive bias matched to the data; a more careful study would construct a dataset where ZZ wins and try to read off what structural feature it exploits.

Hardware experiments. Our noise sweeps used FakeBackend snapshots; running the same circuits on actual IBM hardware would test whether the simulator captures the relevant error channels.

Larger qubit counts. Our barren-plateau experiments stop at 6 qubits; extending to 10–12 qubits (still feasible on classical simulation with optimisation) would let us verify the predicted exponential decay over a larger dynamic range and test whether identity-block initialisation [10] fully mitigates it.

Bias-aware ansätze. Recent work proposes circuit families biased toward translation-invariance, permutation symmetry, or other inductive priors. Comparing such ansätze against generic hardware-efficient ones on appropriately symmetric datasets is an obvious next step.

Variance reduction in training. The MLP and SVM trained in ~ 0.01 –0.5 seconds; the VQC needed 5–60 seconds. A large portion of this is repeated circuit evaluation under COBYLA; substituting gradient-based methods (Adam with parameter-shift gradients) could both speed up training and improve final accuracy.

11 Reproducibility

All code, results, and figures are reproducible from the two scripts `quantum_classifiers.py` and `classical_classifiers.py` included in the directory used to generate the `all_results.csv`. The full sweep takes approximately 2.5 hours on a 2024-era laptop CPU; the headline multi-seed numbers (RQ1, RQ4) can be reproduced in about 30 minutes.

References

- [1] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta. *Supervised learning with quantum-enhanced feature spaces*. *Nature*, 567(7747):209–212, 2019.
- [2] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe. *Circuit-centric quantum classifiers*. *Physical Review A*, 101(3):032308, 2020.
- [3] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. *Quantum circuit learning*. *Physical Review A*, 98(3):032309, 2018.
- [4] M. Schuld. *Supervised quantum machine learning models are kernel methods*. arXiv:2101.11020, 2021.
- [5] J. Preskill. *Quantum computing in the NISQ era and beyond*. *Quantum*, 2:79, 2018.
- [6] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles. *Variational quantum algorithms*. *Nature Reviews Physics*, 3(9):625–644, 2021.
- [7] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven. *Barren plateaus in quantum neural network training landscapes*. *Nature Communications*, 9(1):4812, 2018.
- [8] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles. *Cost function dependent barren plateaus in shallow parametrized quantum circuits*. *Nature Communications*, 12(1):1791, 2021.
- [9] M. Larocca, S. Thanasilp, S. Wang, K. Sharma, J. Biamonte, P. J. Coles, L. Cincio, J. R. McClean, Z. Holmes, and M. Cerezo. *Barren plateaus in variational quantum computing*. *Nature Reviews Physics*, 2025.
- [10] E. Grant, L. Wossnig, M. Ostaszewski, and M. Benedetti. *An initialization strategy for addressing barren plateaus in parametrized quantum circuits*. *Quantum*, 3:214, 2019.
- [11] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre. *Data re-uploading for a universal quantum classifier*. *Quantum*, 4:226, 2020.
- [12] J. M. Kübler, S. Buchholz, and B. Schölkopf. *The inductive bias of quantum kernels*. *Advances in Neural Information Processing Systems*, 34:12661–12673, 2021.
- [13] N. D. Mermin. *Quantum Computer Science: An Introduction*. Cambridge University Press, 2007. (Course textbook “M”.)
- [14] T. G. Wong. *Introduction to Classical and Quantum Computing*. Rooted Grove, 2022. (Course textbook “W”.)

- [15] Qiskit Contributors. *Qiskit: An Open-source Framework for Quantum Computing*. 2024. Version 2.4.1.